

Fragmentation-Aware GPU Cluster Scheduling

A Statistically-Grounded Benchmark, an Improved Algorithm,
and an Honest Account of What It Does and Does Not Fix

Thesis Defense

gputrace + py_sim

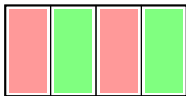
September 12, 2026

The Problem, In One Picture

Node A (4 GPU)



Node B (4 GPU)



4 GPUs idle in aggregate. A job needing 3 contiguous GPUs on one node **fits nowhere**.

Utilization dashboards say 50% free. Usable free capacity is 0.

This Is a Real, Costly, Recent Problem

- GPU-sharing (multiple jobs per physical GPU) is now standard practice
- Fragmentation Gradient Descent (FGD, USENIX ATC '23) was the first scheduler to treat this as a first-class objective
- **FGD is only three years old** – this is a genuinely young research niche, with a lot still unexplored
- Cloud infrastructure keeps finding new ways to fail under resource contention: a 15-hour, 60-country AWS outage in October 2025 traced to a cascading resource issue; a Google auth outage the same year fixed only by *adding capacity*

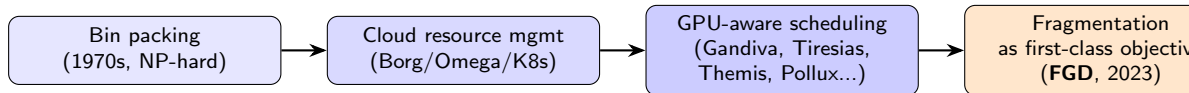
The gap this thesis targets

Every public evaluation of FGD-lineage schedulers we found replays **one fixed historical trace**. A fixed recording cannot contain events that had not yet happened when it was collected.

Thesis Statement

1. A synthetic, statistically-grounded, multi-scenario benchmark is **necessary** to evaluate fragmentation-aware scheduling honestly – a single trace structurally cannot contain the failure modes that matter most.
2. FGD's own fragmentation metric has **two specific, checkable weaknesses** that can be fixed – producing a real, statistically significant improvement over FGD specifically, while **honestly not** claiming a new state of the art against every alternative.

From Bin Packing to GPU Scheduling



The twist

None of the six major GPU-scheduling systems before FGD treats *where, geometrically*, a job lands as its primary objective – placement quality was always a side effect of some other goal.

Two Research Gaps

Gap 1: The metric itself

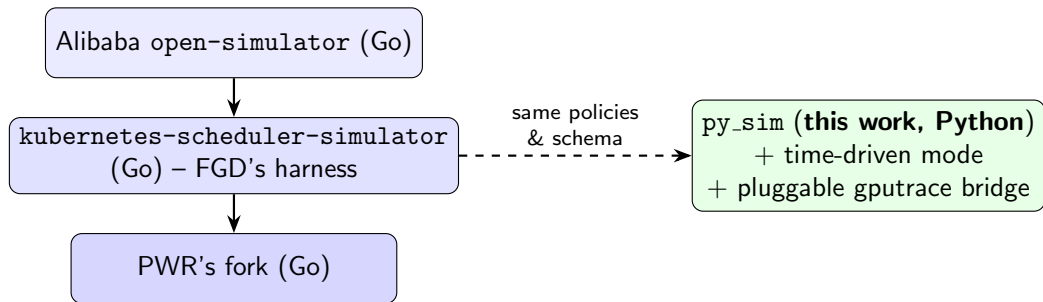
- FGD's 5 representative shapes are weighted **uniformly**
- Each shape's coordinates are **per-dimension quantiles** – can be a “chimera” matching no real job

Gap 2: The evaluation

- Every FGD-lineage paper evaluates on **one fixed trace**
- Real traces are non-stationary (Google, Azure trace studies) – a fixed slice is one draw, not the distribution

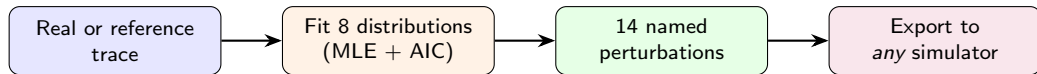
This thesis addresses both.

Simulator Lineage: Why We Rebuilt in Python



- Reimplemented (not forked): first-class Python statistical tooling
- Added a **time-driven** evaluation mode – several stress scenarios are defined entirely by *timing* and invisible to static-mode evaluation alone

gputrace: A Pluggable Stress-Test Generator



Each scenario perturbs one named, specific mechanism

bursty arrivals • GPU-type contention • spot-preemption churn • LLM-inference bursts • MoE expert skew • checkpoint I/O storms • 9 more

Two Checkable Weaknesses in FGD

$$U(n, t) = \frac{1}{|\hat{P}|} \sum_{\hat{p} \in \hat{P}} \mathbb{I}[\neg \text{fits}(\hat{p}, n, t)]$$

Weakness 1: Uniform weighting

All 5 shapes weighted equally, though they represent unequal population mass (10th vs. 50th percentile).

Weakness 2: Chimera shapes

Each shape's cpu/gpu/gpu-count are *independent* marginal quantiles – may match no real job.

Both are provable from FGD's own definition – not assumptions about intent.

w-fgd and w-fgd-balanced

Fix: stratify jobs by demand magnitude, take each stratum's *medoid* (a real job) weighted by population share:

$$\hat{p}_k = \arg \min_{p \in S_k} |m(p) - \text{median}(m(S_k))|, \quad w_k = \frac{|S_k|}{|P|}$$

$$\pi_{\text{w-fgd}}(p, t) = \arg \min_n [U_w(n, t \oplus p) - U_w(n, t)]$$

w-fgd-balanced adds slack-preservation regularization:

$$\pi_{\text{w-fgd-bal}}(p, t) = \arg \min_n [U_w(n, t \oplus p) - U_w(n, t) + \lambda \phi^{\text{gpu}}(n, t \oplus p)]$$

$\lambda = 0$ recovers w-fgd exactly (verified by unit test)

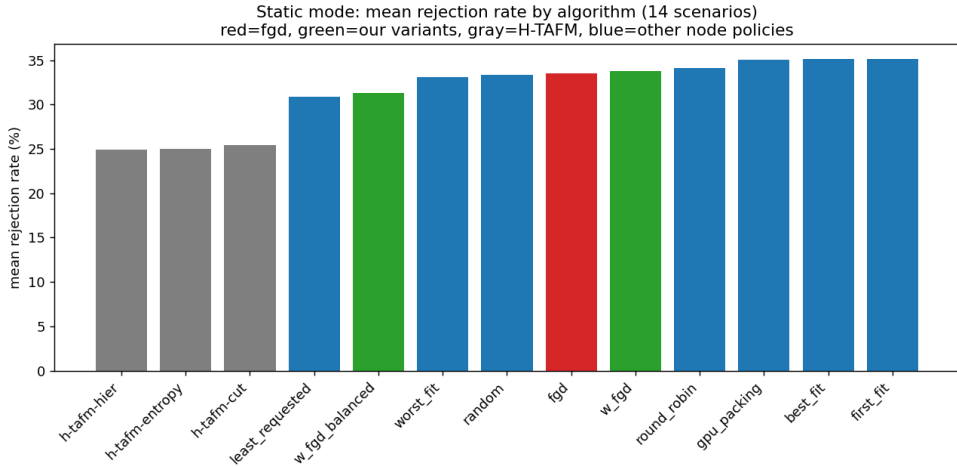
Did We Break FGD? Yes – Specifically

metric (mode)	fgd	w_fgd_balanced	Wilcoxon p
rejection rate (static)	0.3352	0.3133	0.0035
SLA violation (time-driven)	0.1206	0.1022	0.0365
p95 wait time	326.1s	344.1s	0.507 (n.s.)
fairness (Jain)	0.1902	0.1805	0.196 (n.s.)

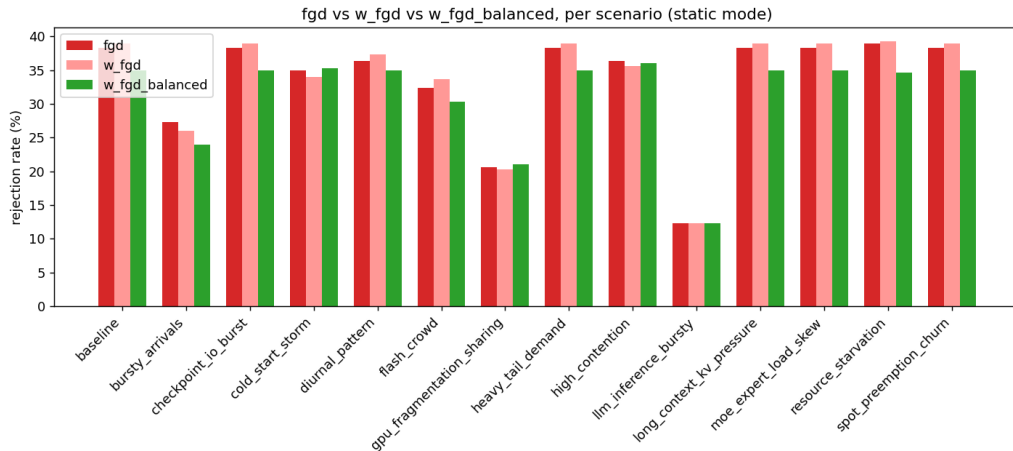
But, reported honestly

Statistically **indistinguishable** from the simplest existing baseline, `least_requested` ($p = 0.81$).

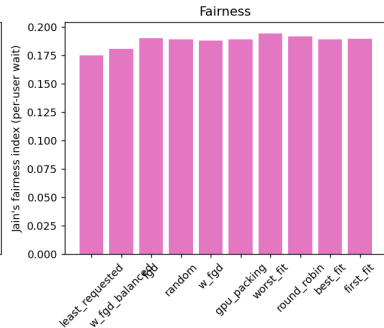
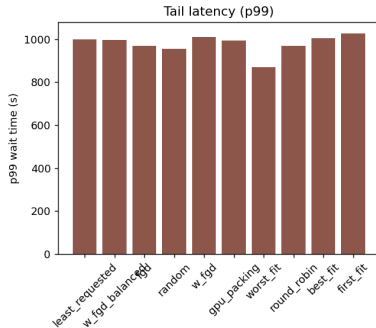
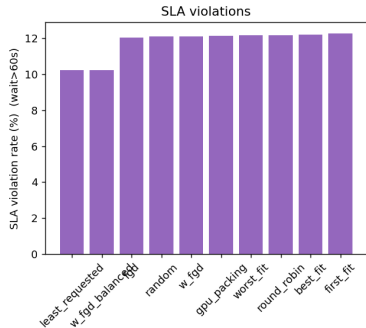
Full Ranking, All 13 Algorithms



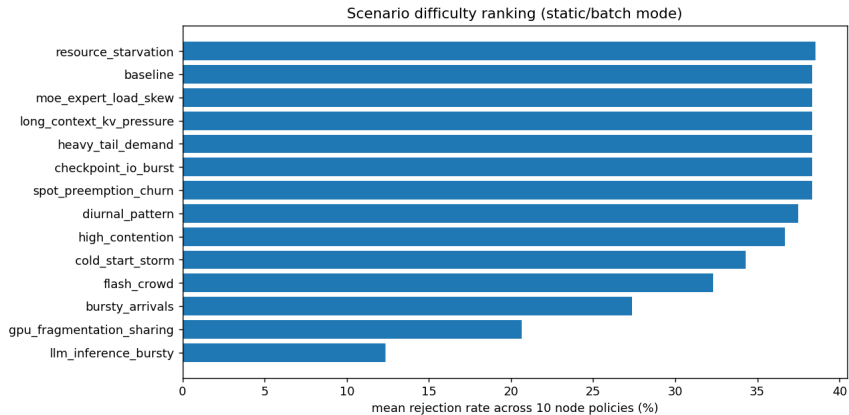
fgd vs. w_fgd vs. w_fgd_balanced, Per Scenario



Broader Metrics: Beyond Rejection Rate



Scenario Difficulty



An Honest Finding About Our Own Generator

scenario	GPU	CPU	duration	submit_time
heavy_tail_demand	✓	✓	✓	✓
checkpoint_io_burst	✓	✓		✓
high_contention	✓	✓	✓	

✓ = identical to baseline, seed 42.

heavy_tail_demand is identical to baseline in *every* column

Its name describes a resource-size change that isn't happening in the current generator – we report this as a bug to fix, not hide it.

What We Actually Showed

- **Built** the missing stress-test instrument (gputrace, 14 scenarios, statistically fit and individually justified)
- **Found** two specific, checkable weaknesses in FGD's metric
- **Fixed** them (w_fgd, w_fgd_balanced) – significant improvement over FGD, SLA violations
- **Reported honestly:** not better than the simplest existing baseline; found a real bug in our own generator; found and fixed a real performance bug in our own simulator

The null result is a finding: the field doesn't yet have a benchmark broad enough to reliably tell real improvement from noise.

Future Directions

- ① Sub-linear pending-queue structure (beyond our constant-factor fix)
- ② Learned λ , not grid-searched
- ③ Disentangle H-TAFM's granularity vs. memory-assumption confounds
- ④ Fix the heavy_tail_demand discrepancy upstream
- ⑤ Scale to production-representative cluster and job counts

Everything reproduces from one archived checkpoint:
`experiments/runs/2026-09-12_fgd_vs_wfgd_h tafm/`

Thank you

https://github.com/anuxlab/py_sim

https://github.com/anuxlab/simulated_data_generation_GPU_Scheduling